

Universidad Tecnológica de La Habana
“José Antonio Echeverría”



Facultad de Ingeniería Informática

DESARROLLO DE UN MÓDULO DE SEGURIDAD POR SOFTWARE PARA LA GESTIÓN DE CLAVES CRIPTOGRÁFICAS Y FIRMA DIGITAL

Autor: Andy Martínez-Aparicio González

Tutora: Dr. C. Lisandra Bravo Ilisástigui
Ing. Rubén Sánchez Rivero

CONTEXTO

En la actualidad, la firma digital se ha consolidado como un mecanismo esencial para garantizar la autenticidad, integridad y no repudio de documentos en entornos digitales.

En Cuba, los últimos años han marcado un creciente énfasis en su incorporación a los servicios digitales del país.

En este contexto la MIPYME INGENIUS crea una plataforma de firma digital llamada eFirma, para satisfacer esta necesidad en el entorno cubano.

SITUACIÓN PROBLEMÁTICA

- Las firmas se realizan de manera local, el usuario debe cargar manualmente su certificado digital en cada operación.
- Esto expone la clave privada fuera de un entorno controlado y aumenta el riesgo de pérdida o robo.
- La clave privada es el activo más crítico de una infraestructura de seguridad, si se compromete se invalida la confianza en todas las firmas emitidas.

CONCEPTOS IMPORTANTES

HSM (Módulo de Seguridad de Hardware)

Dispositivo físico especializado que realiza operaciones criptográficas y custodia claves sin exponerlas al entorno externo.

HSM basado en software

Implementación que simula las funciones del HSM mediante código, ejecutándose sobre servidores convencionales.

PROBLEMA DE INVESTIGACIÓN

¿Cómo desarrollar un módulo de seguridad por software que garantice la protección y gestión de claves criptográficas, así como la ejecución segura de operaciones de firma digital?

OBJETIVO GENERAL

Desarrollar un módulo de seguridad por software que garantice la protección y gestión de claves criptográficas, así como la ejecución segura de operaciones de firma digital.

OBJETIVOS ESPECÍFICOS

1

Identificar los conceptos, normativas y tecnologías asociados a los módulos de seguridad por software y la gestión de claves criptográficas.

2

Diseñar el módulo de seguridad por software a partir de la definición de los requisitos, la arquitectura y el modelo de datos.

3

Implementar el módulo de seguridad por software según el diseño establecido.

4

Validar el módulo desarrollado mediante pruebas unitarias, de integración y de sistema.

TECNOLOGÍAS SELECCIONADAS



Marco de desarrollo:
Spring Boot (Java)



Base de datos:
PostgreSQL

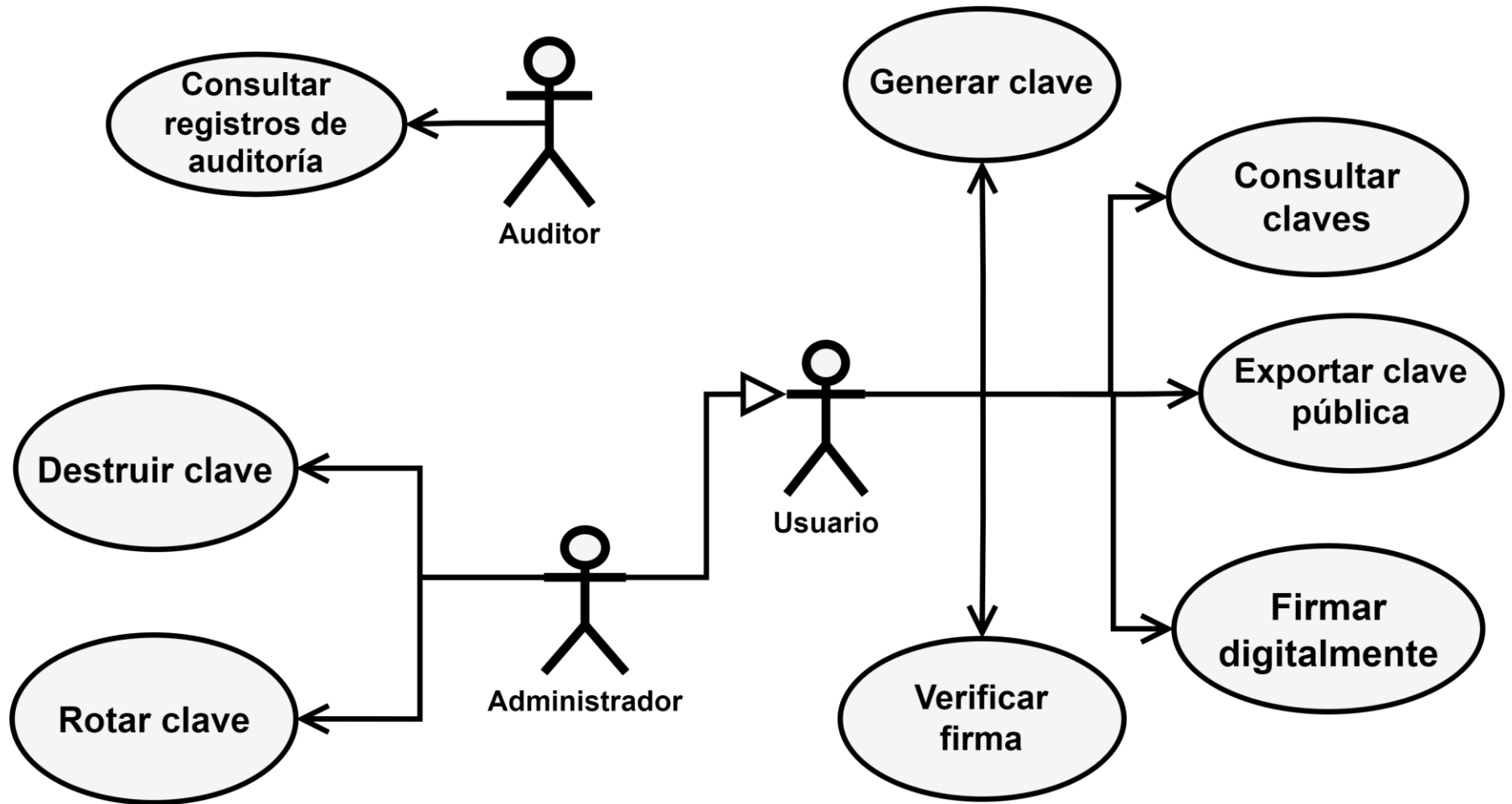


Proveedor de identidad:
Keycloak



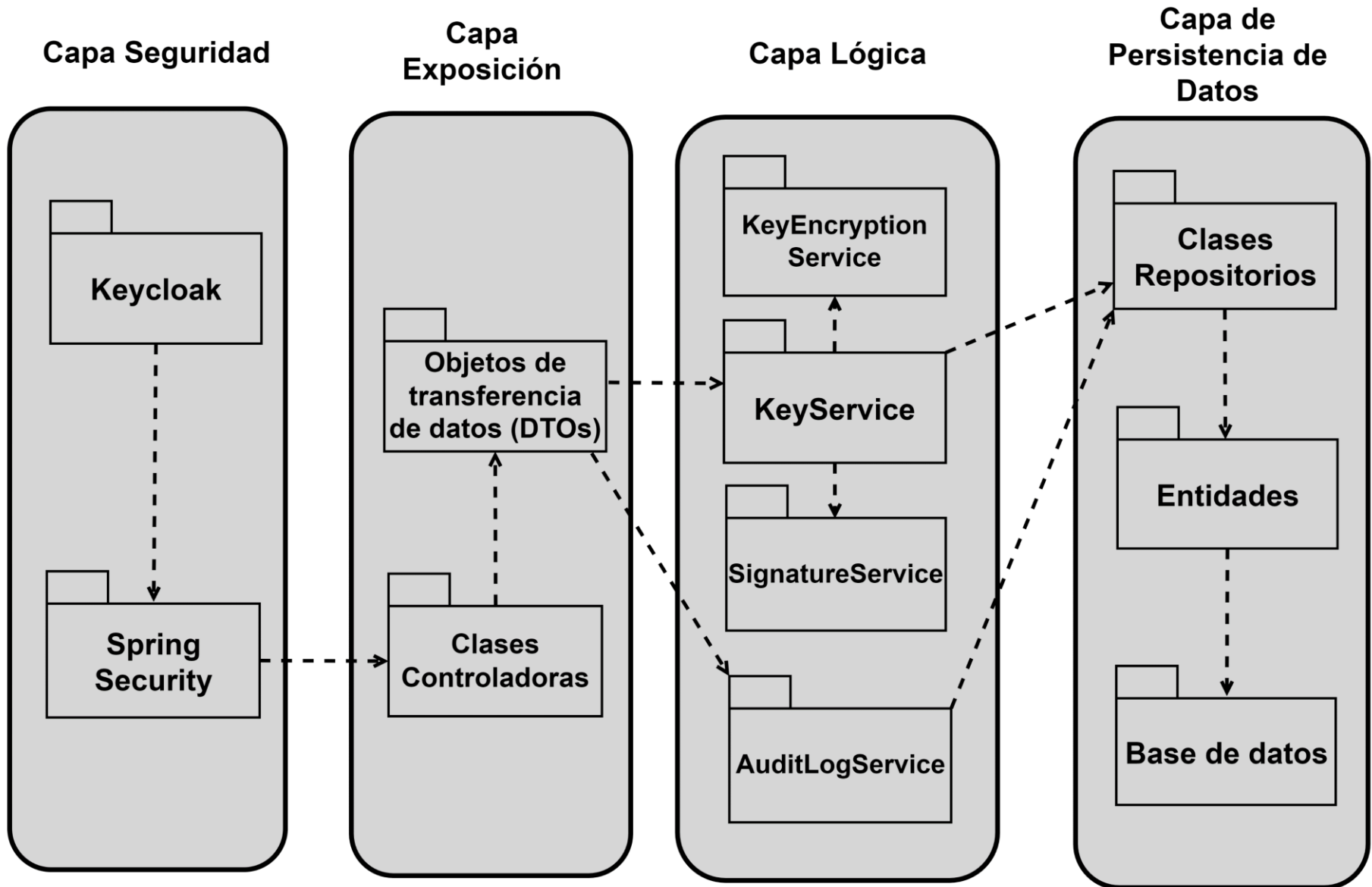
Despliegue:
Docker

DIAGRAMA DE CASOS DE USO DEL SISTEMA

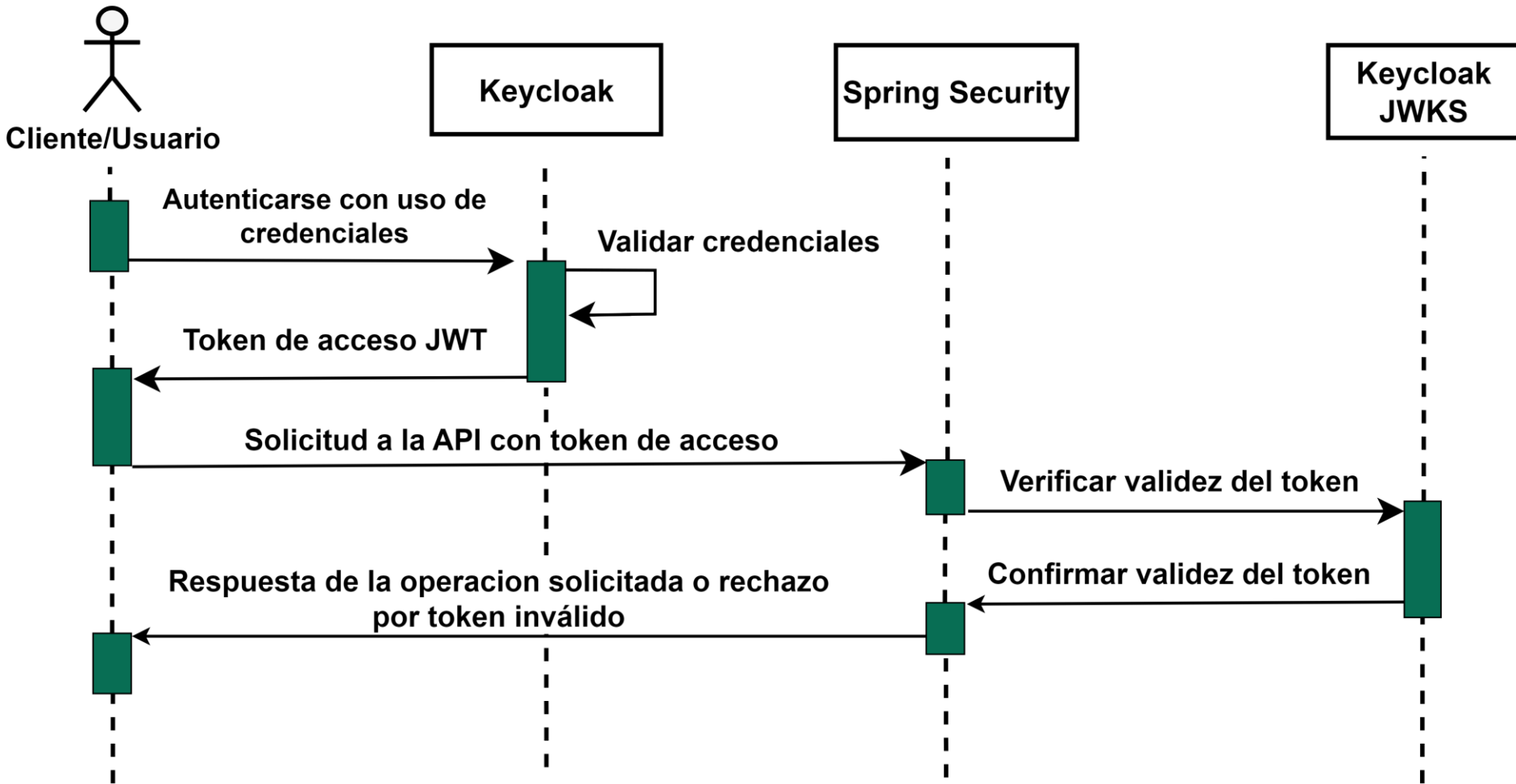


ARQUITECTURA DEL SISTEMA

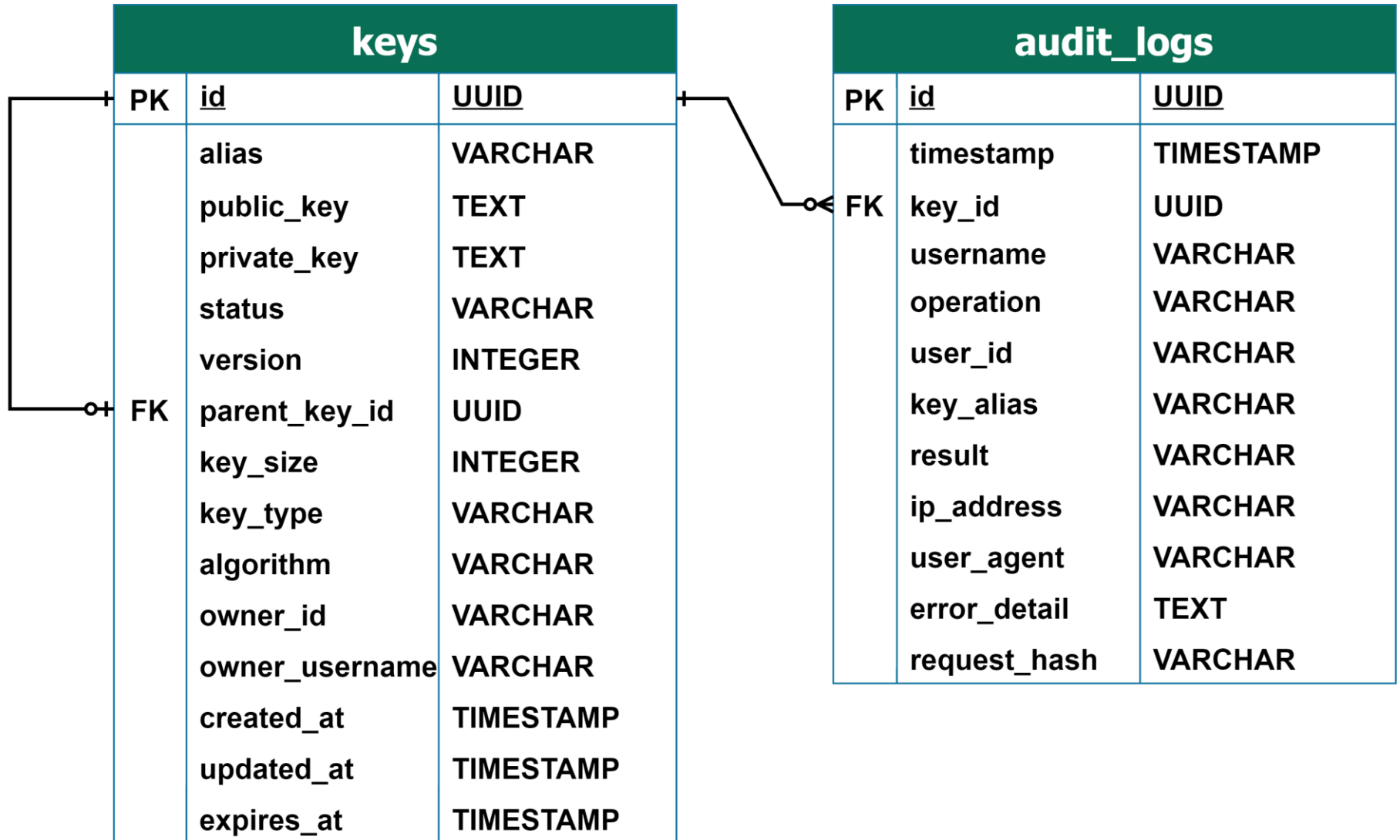
Patrón N-Capas basado en responsabilidades



FLUJO DE AUTENTICACIÓN

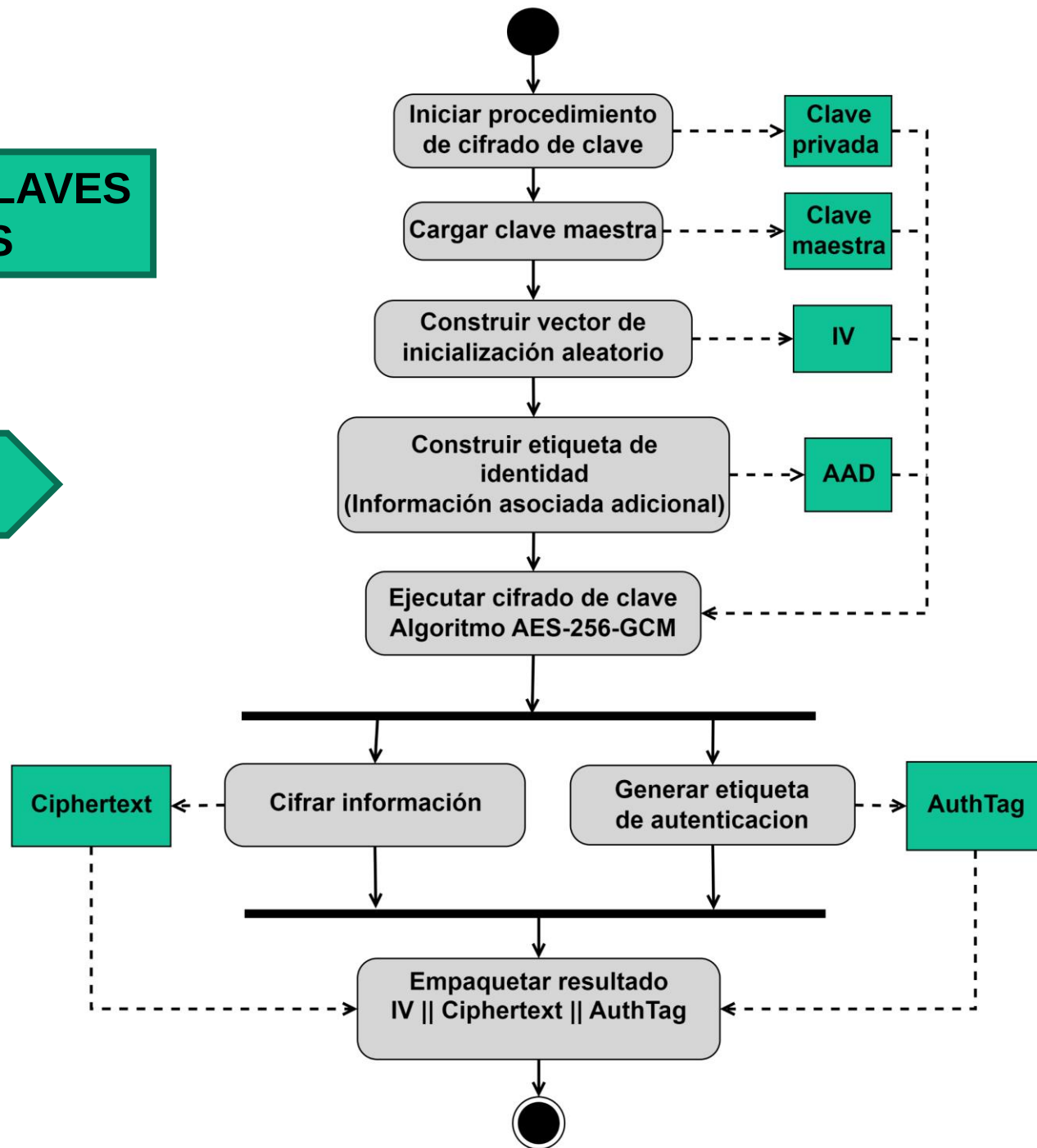


MODELO DE BASE DE DATOS

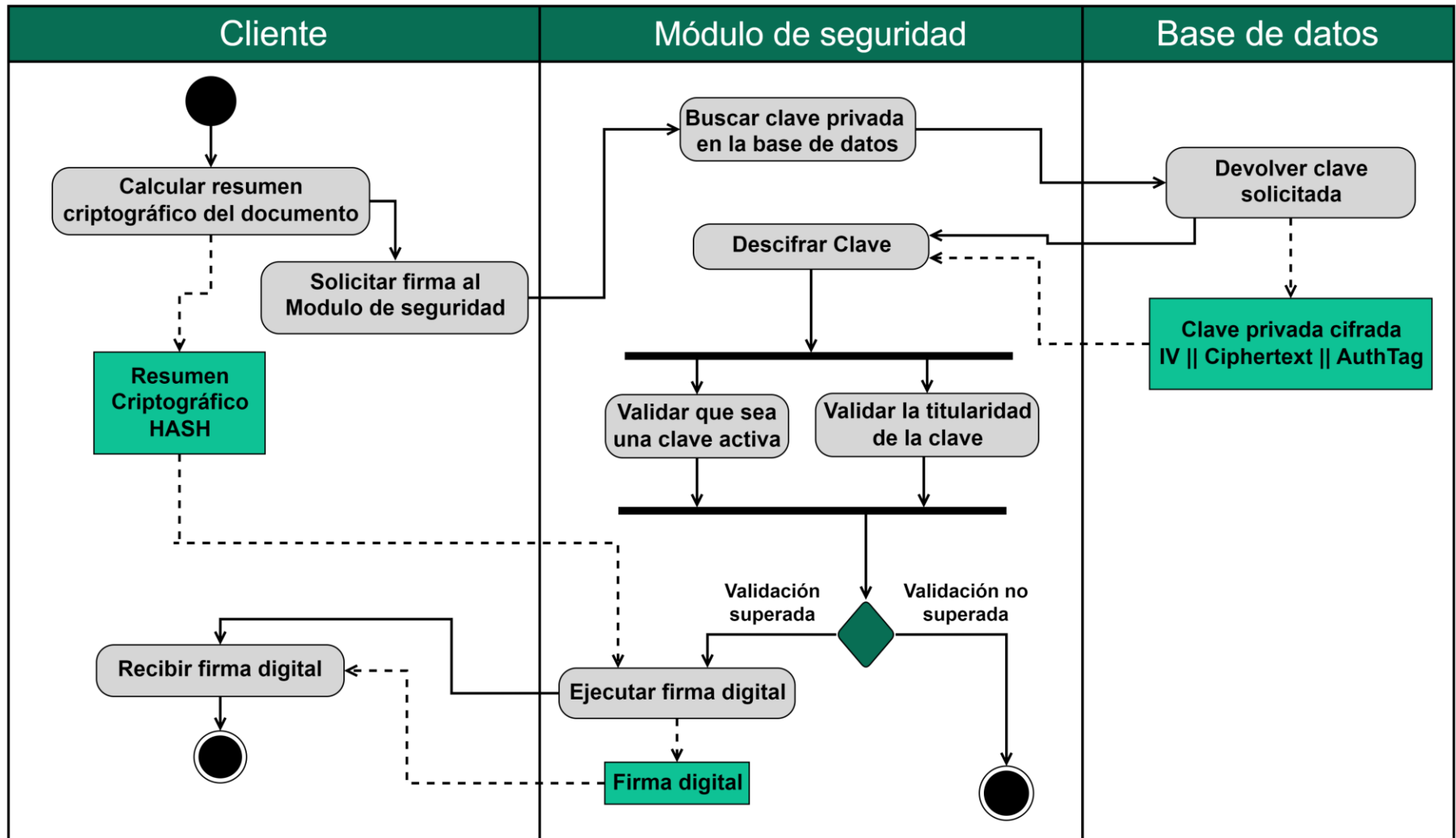


CIFRADO DE CLAVES PRIVADAS

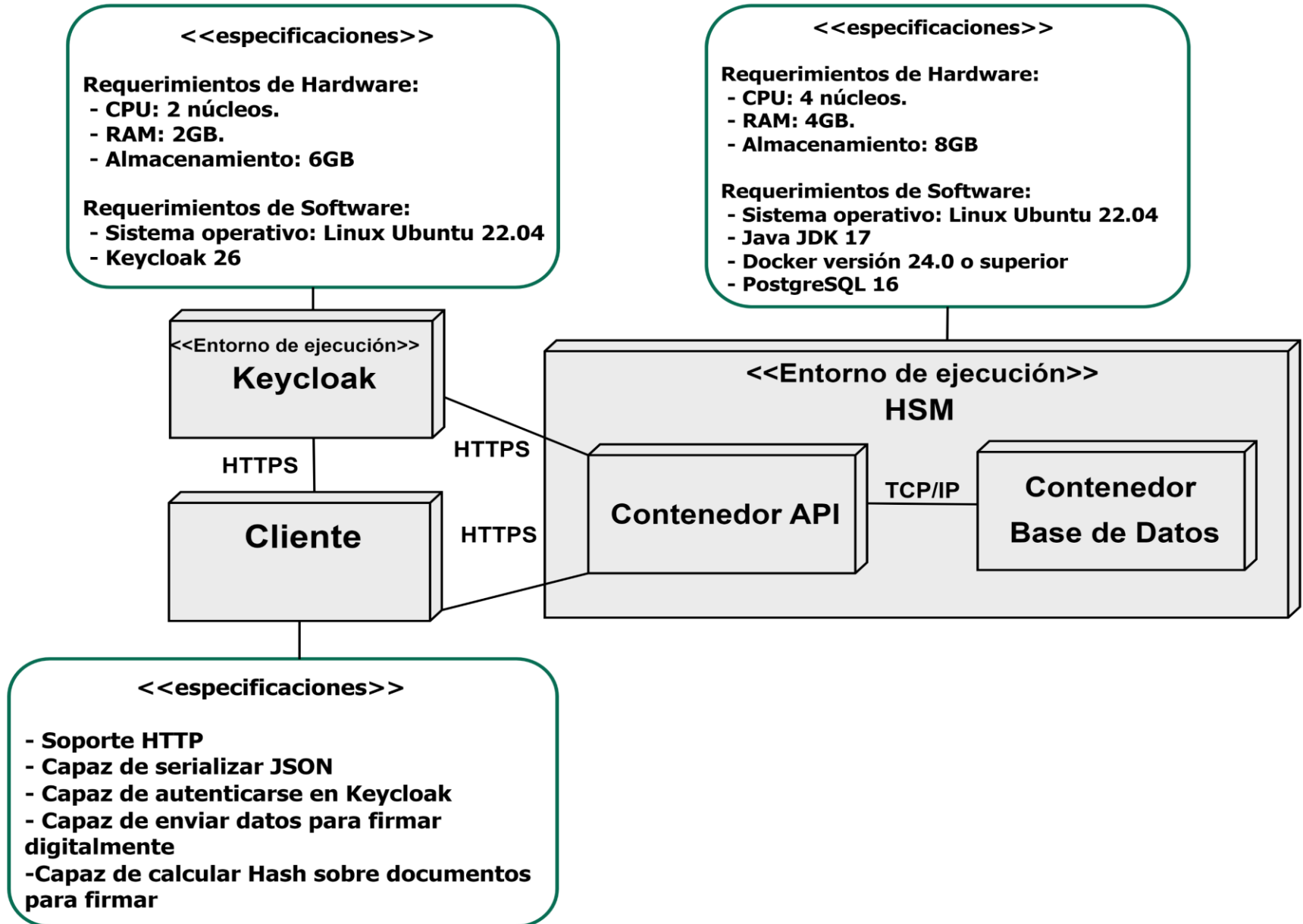
AES-256-GCM



PROCESO DE FIRMA DIGITAL



MODELO DE DESPLIEGUE



ESTRATEGIA DE VALIDACIÓN

PRUEBAS UNITARIAS

Herramientas utilizadas:

- ***JUnit 5***: Entorno de pruebas de Java.
- ***Mockito***: Simula dependencias externas en pruebas aisladas.

Aspectos evaluados:

- Cifrado y descifrado de claves.
- Generación y verificación de firmas.
- Transiciones del ciclo de vida de claves.
- Validaciones de negocio (tamaños, estados).

PRUEBAS UNITARIAS

Iteración	Pruebas ejecutadas	Pruebas fallidas	Causa principal de fallo
1	42	7	• Errores de configuración del entorno de pruebas. • Errores de compatibilidad con la versión de java.
2	42	4	• Manejo incorrecto de excepciones.
3	42	0	-

PRUEBAS DE INTEGRACIÓN

Herramientas utilizadas:

- ***JUnit 5***: Entorno de pruebas de Java.
- ***MockMvc***: Simula peticiones HTTP.
- ***Spring Security Test***: Simula tokens y roles.

Aspectos evaluados:

- Arranque del contexto de la aplicación.
- Creación y consulta de claves vía API REST.
- Firma y verificación digital.
- Control de acceso por roles.
- Flujos HTTP.

PRUEBAS DE INTEGRACIÓN

Iteración	Pruebas ejecutadas	Pruebas fallidas	Causas
1	27	9	• Errores de configuración del entorno de pruebas. • Respuesta esperada incorrecta en el flujo de rotación de claves.
2	27	0	-

PRUEBAS DE SISTEMA

Herramienta utilizada:

- ***Streamlit***: biblioteca de Python para construir interfaces web.

Escenarios:

11

Fallos:

0

Escenarios evaluados:

- Autenticación por roles.
- Gestión de claves.
- Firmas digitales y verificación.
- Consulta de auditoría.

CONCLUSIONES

1

El análisis de los fundamentos teóricos, normativos y tecnológicos permitió establecer el marco conceptual, regulatorio y tecnológico que sustenta el desarrollo de un módulo de seguridad por software para la gestión de claves criptográficas y la firma digital, además de determinar las tecnologías y estándares necesarios para su implementación.

2

Se definió un módulo de seguridad por software sustentado en una arquitectura multicapa, que incorpora una *API REST*, un modelo de datos estructurado y un mecanismo de control de acceso basado en roles, que garantiza niveles adecuados de seguridad, mantenibilidad e interoperabilidad.

CONCLUSIONES

3

A partir del diseño arquitectónico se implementó un sistema criptográfico que integra la gestión del ciclo de vida de claves, la firma digital y un esquema de auditoría segura. La solución incorpora mecanismos de gestión de claves criptográficas, protección de claves privadas, operaciones de firma digital y un registro de auditoría.

4

La validación del sistema mediante la ejecución de pruebas unitarias, de integración y de sistema permitió verificar el correcto funcionamiento de los mecanismos criptográficos y el cumplimiento de los requisitos funcionales y de seguridad.

RECOMENDACIONES

1

Incorporar autenticación de dos factores mediante contraseña de un solo uso (OTP/TOTP) para operaciones críticas como la firma digital, de forma tal que se añada una capa adicional de seguridad más allá de la autenticación por token.

2

Evaluar el rendimiento y la escalabilidad del sistema mediante la ejecución de pruebas de carga y estrés, con el objetivo de determinar los límites operacionales del módulo, la latencia de las operaciones criptográficas bajo condiciones de alta concurrencia y su comportamiento en entornos de producción.

Universidad Tecnológica de La Habana
“José Antonio Echeverría”



Facultad de Ingeniería Informática

DESARROLLO DE UN MÓDULO DE SEGURIDAD POR SOFTWARE PARA LA GESTIÓN DE CLAVES CRIPTOGRÁFICAS Y FIRMA DIGITAL

Autor: Andy Martínez-Aparicio González

Tutora: Dr. C. Lisandra Bravo Ilisástigui
Ing. Rubén Sánchez Rivero

PREGUNTAS DE OPONENCIA

PREGUNTA 1

En un entorno real, ¿cómo asegura que la configuración de producción sea la más restrictiva y que ningún desarrollador pueda desplegar cambios sin aprobación?

CONTROL DE FLUJO DE DESPLIEGUE

Ningún desarrollador debe tener acceso a producción.

Mecanismos de control:

- Revisiones de código obligatorias.
- Aprobaciones manuales de personal autorizado.
- Separación de entornos de desarrollo, pruebas y producción.

PREGUNTA 2

¿Cuál es la principal limitación de su propuesta y cómo podría superarse en trabajos futuros?

FACTOR ÚNICO DE AUTENTICACIÓN

Mecanismo implementado:

Uso de tokens *JWT* emitidos por Keycloak para garantizar identidad y privilegios del usuario.

Recomendación a trabajo futuro:

- Implementación de autenticación de doble factor.
- Aplicación de políticas de expiración de *tokens* más estrictas.
- Implementación de mecanismos de revocación de sesiones activas.

PROTECCIÓN DE LA BASE DE DATOS

Actualmente el control de acceso opera a nivel de aplicación.

Un administrador de la base de datos podría leer datos sensibles y modificar datos.

Recomendación a trabajo futuro:

- Aislar PostgreSQL en una red privada y restringir acceso.
- Implementar *triggers* que bloqueen modificaciones o eliminaciones en las tablas de auditoría.
- Desplegar la extensión *pgAudit* para registrar toda la actividad sobre la base de datos.

PROTECCIÓN DE LA CLAVE MAESTRA

La clave maestra se carga desde una variable de entorno, funcional pero vulnerable ante acceso privilegiado.

Soluciones en la industria:

- Uso de gestores de secretos dedicados. Software especializado que centraliza y protege el almacenamiento
- Uso de un TPM (Módulo de plataforma segura), chip físico, la clave nunca puede extraerse.
- Uso de TEE (Entorno de ejecución confiable), entornos de ejecución aislados dentro del propio procesador.

PREGUNTA 3

¿Cómo aseguró que las bibliotecas criptográficas de terceros, SQLpostgres u otras tecnologías no introducen vulnerabilidades y cómo gestiona su ciclo de actualización en el módulo?

GESTIÓN DE DEPENDENCIAS Y VULNERABILIDADES

- Las operaciones criptográficas se realizan con la extensión nativa de Java, *Java Cryptography Extension* (JCE).
- Se seleccionó PostgreSQL 16 verificando que contara con soporte activo y sin vulnerabilidades conocidas.

- Para las actualizaciones, el proyecto utiliza el gestor de
- dependencias (*spring-boot-starter-parent*). Una sola actualización propaga parches en todas las dependencias.

Universidad Tecnológica de La Habana
“José Antonio Echeverría”



Facultad de Ingeniería Informática

DESARROLLO DE UN MÓDULO DE SEGURIDAD POR SOFTWARE PARA LA GESTIÓN DE CLAVES CRIPTOGRÁFICAS Y FIRMA DIGITAL

Autor: Andy Martínez-Aparicio González

Tutora: Dr. C. Lisandra Bravo Ilisástigui
Ing. Rubén Sánchez Rivero